Laporan Tugas Besar 1

IF3170 Inteligensi Artifisial 2026/2027

Disusun oleh: Kelompok 05 - AlphaIntelligence

Anggota:

Gabriella Botimada Lubis	13524006
An Dafa Anza Avansyah	13524038
Raymond Jonathan Dwi Putra J.	13524059

Dikumpulkan pada: 20 Oktober 2026

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

20 Oktober 2026

Daftar Isi

Daftar Isi **2**
Deskripsi Persoalan **3**
1.1. Deskripsi Permasalahan 3
1.2. Representasi State, Objective Function, dan Constraints 4
Pembahasan **6**
2.1. Implementasi Algoritma 6
2.1.1. Representasi State dan Neighbor 6
2.1.2. Hill-Climbing 7
2.1.2.1. Steepest-Ascent Hill-Climbing 8
2.1.2.2. Stochastic Hill-Climbing 9
2.1.2.3. Hill-Climbing with Sideways Move 10
2.1.2.4. Random-Restart Hill-Climbing 11
2.1.3. Simulated Annealing 12
2.1.4. Genetic Algorithm 14
2.2. Hasil Eksperimen 15
2.2.1. Hasil Eksperimen Hill-Climbing 15
2.2.2. Hasil Eksperimen Simulated Annealing 18
2.2.3. Hasil Eksperimen Genetic Algorithm 23
2.3. Analisis dan Perbandingan 26
2.3.1. Perbandingan Nilai Objective Function 26
2.3.1.1. Hill-Climbing 26
2.3.1.2. Simulated Anneling 26
2.3.1.3. Genetic Algorithm 27
2.3.2. Perbandingan Waktu Eksekusi dan Konsistensi 28
2.3.2.1. Hill-Climbing 28
2.3.2.2. Simulated Annealing 28
2.3.2.3. Genetic Algorithm 29
2.3.3. Pengaruh Parameter Genetic Algorithm 30
2.3.4. Perilaku Pencarian Simulated Annealing 30
Kesimpulan dan Saran **32**
3.1. Kesimpulan 32
3.2. Saran 32
Referensi **33**
Lampiran **34**
Penggunaan Gen-AI 34
Pembagian Tugas 34

Deskripsi Persoalan

1.1. Deskripsi Permasalahan

Keluarga Mejiro berencana memperluas bisnisnya ke bidang logistik darat melalui usaha bernama Mejiro Express. Untuk menjalankan pengiriman percobaan, Mejiro McQueen menawarkan Special Week untuk menjadi pengemudi truk. Namun, Special Week memiliki waktu yang terbatas untuk menentukan paket-paket mana yang perlu untuk dimuat ke dalam peti kemas truk. Setiap paket memiliki ukuran, nilai berat, status barang pecah-belah, dan perkiraan waktu kedatangan yang berbeda. Peti kemas pada truk juga memiliki ukuran ruang dan kapasitas angkut maksimum. Oleh karena itu, Special Week perlu menentukan paket yang akan dimasukkan, posisi penempatan, serta orientasinya agar paket dapat dimuat secara efisien.

Tujuan dari permasalahan ini adalah memperoleh susunan paket dengan total value terbesar di dalam peti kemas dengan menentukan solusi optimal dengan memilih paket yang akan dimuat serta menentukan posisi dan orientasi setiap paket. Tidak semua paket akan dimasukkan ke dalam truk, sehingga paket yang tidak dipilih dianggap berada di luar peti kemas dan tidak memberikan nilai pada solusi.

- Penyusunan paket harus memenuhi beberapa batasan berikut.
- Paket tidak boleh berada di luar atau menembus dinding peti kemas.
- Paket tidak boleh saling bertumpang tindih.
- Paket tidak boleh melayang. Paket yang diletakkan di atas lantai peti kemas harus memiliki penopang dari paket lain.
- Paket berstatus fragile tidak boleh digunakan sebagai penopang bagi paket lain.
- Total berat seluruh paket di dalam truk tidak boleh melebihi kapasitas maksimum peti kemas.

Dalam membantu Special Week menentukan susunan paket yang baik dalam waktu terbatas, program dibuat menggunakan algoritma local search. Algoritma dimulai dari susunan awal yang dibangkitkan secara acak, kemudian mencari solusi yang lebih baik melalui pertukaran dua paket, perpindahan posisi paket, atau rotasi paket sebesar 90 derajat terhadap sumbu koordinat. Setiap solusi dievaluasi berdasarkan total value paket yang berhasil dimuat dengan tetap memenuhi seluruh batasan yang telah ditentukan.

1.2. Representasi State, Objective Function, dan Constraints

Berikut prosedur bagaimana dalam merepresentasikan permasalahan ini:

- Paket hanya berbentuk kubus / balok. Paket dapat berbeda ukuran satu sama lain dan tidak semua harus dimasukkan ke dalam peti kemas pada truk sehingga suatu paket dapat berada di luar **(seluruh bagian paket di luar)**
- Koordinat berupa bilangan bulat. Cara representasi koordinat dibebaskan.
- Struktur data yang digunakan untuk representasi dibebaskan. Misalnya menggunakan *key-value* seperti JSON.
- Inisialisasi (***initial state***) random.
- Langkah neighbor (***move***) yang diperbolehkan tiap iterasi:
    1. Menukar dua paket, baik di dalam maupun di luar truk
    2. Memindahkan sebuah paket ke suatu koordinat yang berbeda
    3. Merotasi paket 90° terhadap sumbu *x*, *y*, atau *z*
- Representasi solusi (***final state***) adalah pemetaan semua paket yang ada dan juga pemetaan semua koordinat dalam peti kemas pada truk. Secara detail yang harus dapat ditunjukkan oleh state dan visualisasi adalah:
    1. Bagaimana keadaan semua paket yang di dalam maupun yang di luar
    2. Bagaimana keadaan peti kemas pada truk
    3. *Value* pada *state* tersebut

Agar permasalahan ini tidak menjadi kompleks, berikut arahan yang diberikan

- **Objective function**
  Permasalahan ini bertujuan untuk memaksimalkan *value* peti kemas pada truk.
- **Kardus tidak bisa melayang dan menembus dinding peti kemas**
  Melayang disini didefinisikan dengan misalkan paket i setelah orientasi tertentu memiliki posisi sudut kiri-bawah-belakang

  $$(x_i, y_i, z_i)$$

  dan dimensi

  $$(w_i, d_i, h_i)$$

  sehingga alas paket i adalah

  $$B_i = \{(x, y, z_i) \mid x_i \leq x < x_i + w_i,\ y_i \leq y < y_i + d_i\})$$

Sebuah titik di alas memiliki *support* apabila:

1. $Z_{i=0}$, artinya di alas truk, atau
2. Terdapat paket lain j yang mempunyai bagian atas tepat di bawah titik tersebut

Maka melayang adalah kondisi tidak ada satupun titik alas yang memiliki support.

- **Barang pecah-belah tidak boleh di bawah paket lain**

  Barang pecah-belah tidak boleh menjadi *support*

- **Truk tidak boleh mengangkut melebihi kapasitasnya**

Pembahasan

2.1. Implementasi Algoritma

2.1.1. Representasi State dan Neighbor

Representasi dasar permasalahan diimplementasikan pada modul `src/core.py` menggunakan kelas-kelas berikut:

Kelas	Fungsi
`Package`	Menyimpan ID, dimensi, value, berat, status fragile, dan ETA paket.
`Truck`	Menyimpan dimensi kontainer dan kapasitas berat maksimum.
`PlacedPackage`	Menyimpan koordinat (x, y, z), orientasi, dan indeks tru tempat paket dimuat.
`State`	Menyimpan katalog paket, data, truk, dan pemetaan ID paket ke penempatannya.

Pemetaan disimpan dalam atribut `placements`. Paket yang ID-nya tidak terdapat pada pemetaan tersebut dianggap berada di luar kontainer. Untuk mendukung fitur bonus *multi-truck*, `State` menyimpan truk utama melalui atribut `truck` dan truk tambahan melalui `additional_trucks`.

```
@dataclass
class State:
    truck: Truck
    packages: Tuple[Package, ...]
    placements: Dict[str, PlacedPackage]
    additional_trucks: Tuple[Truck, ...] = ()
```

State awal dibentuk oleh `generate_initial_state()`. Fungsi ini mengacak urutan paket, orientasi, dan koordinat penempatan, kemudian mempertahankan penempatan yang valid. Apabila percobaan acak gagal menempatkan suatu paket, fungsi mencoba mencari posisi valid pada lantai kontainer. Pada multi-truck, truk diisi secara bergantian dengan paket yang belum dimuat.

Pembentukan neighbor untuk Hill-Climbing dan Simulated Annealing menggunakan tiga jenis operasi:

1. Pertukaran paket, melalui `swap_packages()`, termasuk pertukaran paket di dalam dengan paket di luar kontainer.
2. Pemindahan paket, melalui `move_package()` dan `_random_move()`, termasuk memasukkan, mengeluarkan, atau memindahkan paket ke truk lain.
3. Rotasi paket, melalui `rotate_package()`, berupa rotasi 90° terhadap sumbu `x`, `y`, atau `z` pada paket yang sedang dimuat.

Fungsi `generate_neighbor()` menghasilkan satu neighbor valid, sedangkan `generate_neighbors()` menghasilkan sekumpulan neighbor valid yang tidak duplikat melalui pengambilan sampel acak. Pemeriksaan dilakukan menggunakan `validate_state()` atau `is_valid_state()`, meliputi batas kontainer, overlap, support, fragile, dan kapasitas berat.

Nilai state dihitung oleh `calculate_objective()` sebagai jumlah value seluruh paket yang dimuat. Pada multi-truck, penjumlahan dilakukan untuk seluruh armada, sedangkan batas kapasitas dan geometri diperiksa pada masing-masing truk.

2.1.2. Hill-Climbing

Algoritma Hill-Climbing diimplementasikan pada `src/hill_climbing.py` dengan implementasi yang dibuat, diantaranya Steepest-Ascent, Stochastic Hill-Climbing, Hill-Climbing with Sideways Move, dan Random-Restart Hill-Climbing. Setiap algoritma menerima state awal yang valid, mengevaluasi solusi dengan fungsi `calculate_objective()`, lalu menghasilkan objek SearchResult. Objek tersebut menyimpan state awal dan akhir, riwayat objective, jumlah iterasi, dan durasi pencarian. Proses algoritma yang dibuat dilakukan pada proses yang sama oleh `src/main.py` dan disimpan oleh `src/output_manager.py`. State awal untuk Steepest-Ascent, Stochastic Hill-Climbing, dan Hill-Climbing with Sideways Move dibangkiatkan oleh `generate_initial_state()` menggunakan seed dari input. Setelah terbentuk, setiap kandidat diperiksa terhadap batas kontainer, overlap, support, fragile, dan kapasitas berat. Sedangkan Random-Restart Hill-Climbing, state awal baru dibangkitkan pada setiap restart dengan seed turunan dari seed utama.

Secara umum, tahapan Hill-Climbing yang digunakan adalah sebagai berikut.

1. Program membangkitkan state awal yang valid secara acak.
2. Program membangkitkan neighbor melalui pertukaran paket, pemindahan paket, atau rotasi paket.

3. Setiap neighbor diperiksa validitasnya dan dievaluasi menggunakan objective function.
4. Algoritma memilih state berikutnya sesuai aturan varian Hill-Climbing yang digunakan.
5. Pencarian berhenti ketika tidak ada perbaikan yang dapat dilakukan atau batas parameter algoritma telah tercapai.
6. State akhir dan data eksperimen disimpan untuk ditampilkan dalam ringkasan serta visualisasi.

Tiga operasi neighbor yang dilakukan memiliki ketentuan permasalahan, yaitu pertukaran dua paket, pemindahan paket, dan rotasi paket. Sebelum neighbor digunakan, state hasil operasi divalidasi terlebih dahulu dengan mengecek apakah memenuhi dengan ketentuan yang ada. Untuk algoritma yang membutuhkan seluruh tetangga, fungsi `generate_all_neighbors()` seluruh state valid yang dapat dicapai oleh satu operasi. Sementara itu, `generate_neighbors()` membangkitkan satu tetangga valid secara acak untuk Stochastic Hill-Climbing.

2.1.2.1. Steepest-Ascent Hill-Climbing

Steepest-Ascent Hill-Climbing melakukan evaluasi pada seluruh neighbour valid dari state saat ini dan melakukan seleksi pada neighbour dengan mengambil nilai objective terbesar dari state saat ini sebagai state berikutnya. Pencarian berhenti apabila tidak ada neighbour yang dievaluasi memberikan peningkatan atau ketika jumlah interasi mencapai max_iterations. Algoritma Hill-Climbing ini selalu memperbaiki nilai solusi yang ada, namun dapat berhenti pada local optimum.

```
def steepest_ascent(initial_state: State, max_iterations: int
= 100,
    seed: Optional[int] = None) -> SearchResult:
    start = time.perf_counter()
    current = initial_state.copy()
    history = [calculate_objective(current)]

    for _ in range(max_iterations):
        neighbors = generate_all_neighbors(current)
        if not neighbors:
            break

        best_neighbor = None
        best_score = history[-1]

        for neighbor in neighbors:
            neighbor_score = calculate_objective(neighbor)
            if neighbor_score > best_score:
```

```
                best_neighbor = neighbor
                best_score = neighbor_score

        if best_neighbor is None:
            break

        current = best_neighbor
        history.append(best_score)

    return _result("steepest_ascent", initial_state, current,
history,iteration_count=len(history) - 1, start=start,
seed=seed)
```

2.1.2.2. Stochastic Hill-Climbing

Stochastic Hill-Climbing dilakukan dengan membangkitkan tepat satu neighbor secara acak pada setiap iterasi dilakukan. State berpindah hanya apabila neighbor tersebut memiliki nilai objective lebih tinggi dari state saat ini. ika neighbor tidak lebih baik, state saat ini dipertahankan dan iterasi berikutnya tetap dijalankan sampai batas max_iterations tercapai. Seed digunakan agar urutan neighbor acak dapat direproduksi pada pengujian yang sama.

```
def stochastic_hill_climbing(initial_state: State,
max_iterations: int = 100,
    seed: Optional[int] = None) -> SearchResult:
    start = time.perf_counter()
    rng = random.Random(seed)
    current = initial_state.copy()
    history = [calculate_objective(current)]

    for _ in range(max_iterations):
        neighbor_seed = rng.randrange(1_000_000_000)
        neighbor = generate_neighbor(current,
seed=neighbor_seed)
        if neighbor is None:
            break
        if calculate_objective(neighbor) >
calculate_objective(current):
            current = neighbor
        history.append(calculate_objective(current))

    return _result("stochastic_hill_climbing", initial_state,
current, history, iteration_count=len(history) - 1,
start=start, seed=seed)
```

2.1.2.3. Hill-Climbing with Sideways Move

Hill-Climbing with Sideways Move memiliki prosedur yang pemilihan neighbour terbaik sama seperti Steepest-Ascent, namun memiliki tambahan pemilihan, yaitu algoritma ini dapat berpindah ke neighbor dengan nilai objective yang sama ketika tidak ada peningkatan. Jumlah perpindahan tanpa peningkatan dibatasi melalui parameter `max_sideways_moves` untuk mencegah algoritma berputar terlalu lama pada plateau. Pada pembuatannya, nilai parameter tersebut ditetapkan menjadi 10.

```
def hill_climbing_sideways(initial_state: State,
max_iterations: int = 100,
    max_sideways_moves: int = 10, seed: Optional[int] = None)
-> SearchResult:
    rng = random.Random(seed)
    current = initial_state.copy()
    history = [calculate_objective(current)]
    sideways_moves = 0

    for _ in range(max_iterations):
        neighbors = generate_all_neighbors(current)
        if not neighbors:
            break

        best_score = None
        choices = []
        for neighbor in neighbors:
            score = calculate_objective(neighbor)
            if best_score is None or score > best_score:
                best_score = score
                choices = [neighbor]
            elif score == best_score:
                choices.append(neighbor)

        if best_score > history[-1]:
            current = rng.choice(choices)
            history.append(best_score)
            sideways_moves = 0
        elif best_score == history[-1] and sideways_moves <
max_sideways_moves:
            current = rng.choice(choices)
            history.append(best_score)
            sideways_moves += 1
        else:
            break
```

2.1.2.4. Random-Restart Hill-Climbing

Random-Restart Hill-Climbing melakukan proses seperti Steepest-Ascent Hill Climbing berulang kali dari state awal acak yang berbeda. Hasil akhir ditentukan berdasarkan hasil terbaik dari seluruh pencarian yang telah dilakukan. Parameter `max_restarts = 5` digunakan untuk melakukan satu pencarian awal dan lima restart tambahan, sehingga terdapat enam proses pencarian Steepest-Ascent pada setiap run. Program mencatat objective akhir dan jumlah iterasi dari setiap pencarian agar hasil restart dapat diperiksa kembali.

```
def random_restart_hill_climbing(truck: Truck | Tuple[Truck,
...],
    packages: Iterable[Package], max_restarts: int = 5,
    max_iterations: int = 100, seed: Optional[int] = None) ->
SearchResult:
    rng = random.Random(seed)
    package_tuple = tuple(packages)
    best_result = None
    iterations_per_restart = []
    objectives_per_restart = []

    for _ in range(max_restarts + 1):
        initial_seed = rng.randrange(1_000_000_000)
        search_seed = rng.randrange(1_000_000_000)
        initial = generate_initial_state(truck,
package_tuple, seed=initial_seed)
        result = steepest_ascent(initial,
max_iterations=max_iterations,
            seed=search_seed)
        iterations_per_restart.append(result.iterations)

objectives_per_restart.append(calculate_objective(result.fina
l_state))

        current_score =
calculate_objective(result.final_state)
        if best_result is None:
            best_result = result
            continue

        best_score =
calculate_objective(best_result.final_state)
        if current_score > best_score:
            best_result = result
```

2.1.3. Simulated Annealing

Simulated Annealing diimplementasikan pada modul src/simulated_annealing.py. Algoritma menerima satu *initial state* valid yang dibangkitkan secara acak oleh generate_initial_state(). Implementasi algoritma ini menggunakan beberapa parameter utama, yaitu initial_temperature, cooling_rate, min_temperature, dan max_iterations. Selain itu, terdapat stuck_threshold dan local_optimum_probe_size untuk mendeteksi kejadian *stuck* pada local optimum. Adapun parameter seed yang digunakan agar pembangkitan *state* dan *neighbor* acak dapat direproduksi.

Tahapan algoritma adalah sebagai berikut:

1. State awal disalin menjadi current dan best. Nilai objective state tersebut dihitung sebagai current_score dan best_score, sedangkan temperatur diinisialisasi dengan initial_temperature.
2. Pada setiap, fungsi generate_neighbor() membangkitkan satu neighbor valid secara acak. Jika tidak ditemukan neighbor valid, pencarian dihentikan.
3. Nilai perubahan *objective* dihitung dengan rumus
$$\Delta E = objective(neighbor) - objective(current)$$
Neighbor dengan $\Delta E \geq 0$ selalu diterima, sedangkan neighbor dengan $\Delta E < 0$ dapat diterima dengan probabilitas
$$P = e^{\Delta E/T}$$
dengan T sebagai temperatur saat ini.
4. Jika *neighbor* diterima, *neighbor* menjadi current state. Jika *objective current* tersebut melebihi best_score, state disimpan sebagai best_state. Penyimpanan best_state dilakukan karena Simulated Annealing dapat menerima state yang lebih buruk sehingga state akhir belum tentu merupakan state terbaik.
5. Frekuensi stuck dideteksi menggunakan no_improvement_streak. Jika tidak ditemukan peningkatan terhadap best_score selama stuck_threshold iterasi, algoritma mengambil sampel *neighbor* melalui generate_neighbor() sebanyak maksimum local_optimum_probe_size. Apabila tidak terdapat *neighbor* dengan *objective* lebih tinggi daripada *current state*, nilai stuck_count akan di-*increment* dan nomor iterasinya disimpan pada *stuck iterations*. Satu episode *stuck* dihitung sekali hingga algoritma kembali menemukan *best objective* yang baru.
6. Pada akhir tiap iterasi, temperatur diturunkan secara geometrik menggunakan rumus
$$T_{baru} = T_{lama} \times cooling_rate$$

Proses dihentikan ketika jumlah iterasi mencapai max_iterations, temperatur mencapai min_temperature, atau tidak ditemukan *neighbor* valid.

Berikut merupakan kode inti proses penerimaan neighbor dan pendinginan temperatur:

```
neighbor = generate_neighbor(current, seed=neighbor_seed)
if neighbor is None:
    stop_reason = "no_valid_neighbor"
    break

neighbor_score = calculate_sa_objective(neighbor,
objective_mode)
delta_e = neighbor_score - current_score

if delta_e >= 0:
    acceptance_probability = 1.0
    accept = True
else:
    acceptance_probability = math.exp(delta_e / temperature)
    accept = rng.random() < acceptance_probability

if accept:
    current = neighbor
    current_score = neighbor_score

    if delta_e < 0:
        accepted_worse_moves += 1

if current_score > best_score:
    best = current.copy()
    best_score = current_score

temperature *= cooling_rate
```

Selain objective utama ‘value’, implementasi algoritma Simulated Annealing menyediakan objective mode lain yaitu ‘eta_weighted’. Mode ‘value’ menggunakan calculated_objective() yang menghitung nilai objective dengan menjumlahkan value seluruh paket di dalam truk. Sementara itu, mode’eta_weighted’ memanfaatkan atribut ETA yang tersedia dengan rumus

$$f(S) = \sum_{i \epsilon S} value_i (1 + \frac{1}{ETA_i + 1})$$

sehingga paket dengan ETA yang lebih kecil memperoleh bobot lebih besar dengan tetap mempertimbangkan value paket. Pemilihan kedua mode tersebut dilakukan oleh fungsi calculate_sa_objective():

```
def calculate_sa_objective(state,
objective_mode=OBJECTIVE_VALUE):
    if objective_mode == OBJECTIVE_VALUE:
        return float(calculate_objective(state))

    if objective_mode == OBJECTIVE_ETA_WEIGHTED:
        return calculate_eta_weighted_objective(state)

    raise ValueError(
        "objective_mode must be 'value' or 'eta_weighted'"
    )
```

Hasil pencarian disimpan dalam SimulatedAnnealingResult yang memuat initial_state, final_state, best_state, riwayat objective, temperature, ΔE, probabilitas penerimaan, *neighbor* yang diterima, jumlah iterasi, durasi, serta statistik *stuck*. Riwayat tersebut digunakan oleh src/output_manager.py dan src/visualizatiom.py untuk menghasilkan state awal dan akhir, grafik *objective* terhadap iterasi, informasi penerimaan *neighbor*, durasi pencarian, serta frekuensi *stuck* yang diperlukan dalam eksperimen.

2.1.4. Genetic Algorithm

Genetic Algorithm diimplementasikan pada `src/genetic_algorithm.py` dengan parameter utama `population_size` dan `max_iterations`. Kelas `ChromosomeCodec` merepresentasikan setiap individu sebagai satu string digit desimal. Setiap paket memiliki blok berisi (`truck_code`, `x`, `y`, `z`, `orientation`), dengan lebar *field* tetap untuk satu *input*. Kode truk nol menunjukkan bahwa paket di luar kontainer, sedangkan orientasi menggunakan indeks 0–5.

Tahapan algoritma adalah sebagai berikut:

1. `initialize_population()` membentuk populasi state acak yang valid, kemudian mengubahnya menjadi kromosom.
2. `evaluate_population()` melakukan decoding dan validasi. Fitness value dihitung sebagai total value paket yang dimuat, termasuk seluruh truk pada multi-truck.

3. `roulette_wheel_selection()` memilih orang tua dengan peluang proporsional terhadap fitness Jika seluruh fitness ol, pemilihan dilakukan secara seragam.
4. `one_point_crossover()` menggabungkan dua orang tua pada satu batas antarblok paket untuk menghasilkan satu anak.
5. `mutate()` memiliki probabilitas 0,1 per anak untuk mengubah satu field. Mutasi orientasi dilakukan melalui rotasi 90° terhadap sumbu acak.
6. `repair_chromosome()` memperbaiki penempatan dengan penyusunan dari bawah, prioritas value, dan percobaan reinserasi paket yang ditolak. Paket yang tetap tidak dapat ditempatkan dinyatakan berada di luar kontainer. Repair ini merupakan heuristik tambahan untuk menjaga validitas solusi.

Berikut adalah kode inti pembentukan generasi pada `next_generation()`:

```
fitness_scores, _, _ = evaluate_population(population, codec)
children = []

while len(children) < len(population):
    parent_a = roulette_wheel_selection(population,
fitness_scores, rng)
    parent_b = roulette_wheel_selection(population,
fitness_scores, rng)
    child = one_point_crossover(parent_a, parent_b, codec,
rng)
    child = mutate(child, codec, rng)
    children.append(repair_chromosome(child, codec))

return children
```

2.2. Hasil Eksperimen

2.2.1. Hasil Eksperimen Hill-Climbing

Pengujian Hill-Climbing dilakukan dengan satu truk berukuran 8 x 8 x 5 dengan kapasitas berat 40, dan 12 paket. Setiap varian dijalankan tiga kali menggunakan seed 42, 123, dan 999. Nilai `max_iterations` untuk semua varian Hill-Climbing adalah 100. Khusus Hill-Climbing with Sideways Move, `max_sideways_moves` bernilai 10 dan khusus Random-Restart Hill-Climbing, `max_restarts` bernilai 5. Durasi pada tabel hanya mencakup proses pencarian. Proses pembacaan input, penulisan JSON atau CSV, serta pembuatan visualisasi tidak termasuk dalam pengukuran waktu. Seluruh state akhir pada tabel telah melewati validasi constraint.

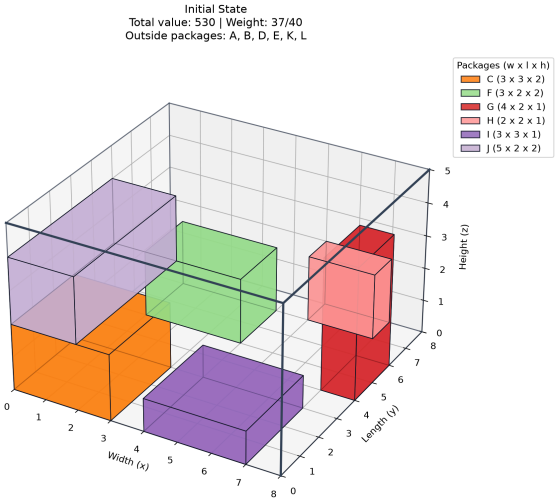
Algoritma	Seed	Objective awal	Objective akhir	Iterasi	Durasi (s)	Keterangan
Steepest-Ascent	42	530	575	1	0,195	-
Steepest-Ascent	123	525	560	2	0,212	-
Steepest-Ascent	999	585	590	1	0,244	-
Stochastic	42	530	575	100	0,015	-
Stochastic	123	525	560	100	0,010	-
Stochastic	999	585	590	100	0,019	-
Sideways Move	42	530	575	11	1,201	10 sideways move
Sideways Move	123	525	560	12	0,936	10 sideways move
Sideways Move	999	585	590	11	1,503	10 sideways move
Random Restart	42	570	580	5	1,500	5 restart, 6 pencarian
Random Restart	123	585	585	5	1,608	5 restart, 6 pencarian
Random Restart	999	580	590	5	1,482	5 restart, 6 pencarian

Ringkasan rata-rata tiga pengulangan ditunjukkan pada tabel berikut. Simpangan baku dihitung dari objective akhir ketiga run dan digunakan hanya untuk melihat variasi hasil pada eksperimen ini.

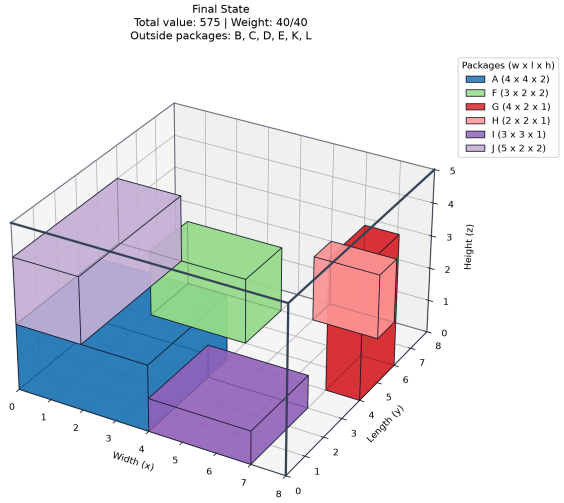
Algoritma	Rata-rata objective akhir	Simpangan baku	Rata-rata iterasi	Rata-rata durasi (s)
Steepest-Ascent	575,00	12,25	1,33	0,217

Stochastic	575,00	12,25	100,00	0,014
Sideways Move	575,00	12,25	11,33	1,213
Random Restart	585,00	4,08	5,00	1,530

Setiap run menghasilkan state awal, state akhir, dan grafik objective terhadap iterasi. Berikut gambar dari salah satu pada algoritma steepest ascent Hill-Climbing dengan seed 42, untuk output lainnya dapat dilihat pada path berikut `output/hill_climbing`.



Gambar initial state pada algoritma steepest ascent Hill-Climbing



Gambar final state pada algoritma steepest ascent Hill-Climbing



2.2.2. Hasil Eksperimen Simulated Annealing

Pengujian utama Simulated Annealing dilakukan menggunakan *test case single-truck* yang sama dengan algoritma lainnya, yaitu satu truk berukuran 8 x 8 x 5 dengan kapasitas berat maksimum 40 dan 12 paket. Setiap konfigurasi dijalankan sebanyak tiga kali menggunakan *seed* 42, 123, dan 999, di mana penggunaan *seed* yang berbeda menghasilkan initial state dan jalur pencarian yang berbeda.

Berikut merupakan parameter Simulated Annealing yang dibuat tetap pada seluruh percobaan.

Initial temperature	: 100
Cooling rate	: 0.99
Minimum temperature	: 0.001
Maximum iterations	: 1000
Stuck threshold	: 50
Local optimum probe size	: 50

Eksperimen dilakukan terhadap dua mode *objective*, yaitu value sebagai objective utama dan eta_weighted sebagai *objective* tambahan.

Hasil Objective Utama (value)

Pada mode value, hasil ketiga percobaan ditunjukkan pada tabel berikut.

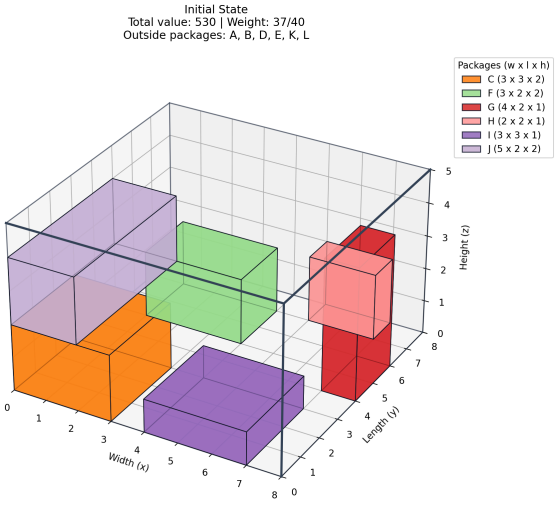
Run	Seed	Objective Awal	Objective Akhir	Objective Terbaik	Accepted Worse Moves	Stuck	Iterasi Stuck	Durasi
1	42	530	575	575	20	2	155, 239	9.862
2	123	525	575	575	16	2	179, 309	4.657
3	999	585	560	585	15	1	281	23.289
Rata-rata		**546.67**	**570.00**	**578.33**	**17.00**	**1.67**	**–**	**12.603**

Sebagai ringkasan perubahan state, paket yang dimuat pada state awal dan akhir adalah sebagai berikut. Informasi lengkap mengenai posisi, orientasi, serta paket yang berada di luar truk disimpan pada states.json dan visualisasi setiap percobaan.

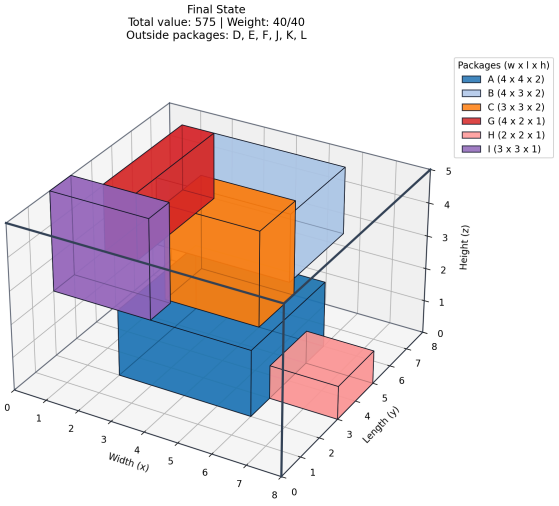
Run	State Awal	State Akhir

1	C, F, G, H, I, J	A, B, C, G, H, I
2	C, D, F, H, J	A, F, G, H, I, J
3	A, C, E, F, G, H, L	A, C, D, G, I

Untuk menunjukkan perubahan konfigurasi secara visual, state awal dan akhir Run 1 dapat ditunjukkan sebagai berikut.



Gambar initial state pada algoritma simulated annealing

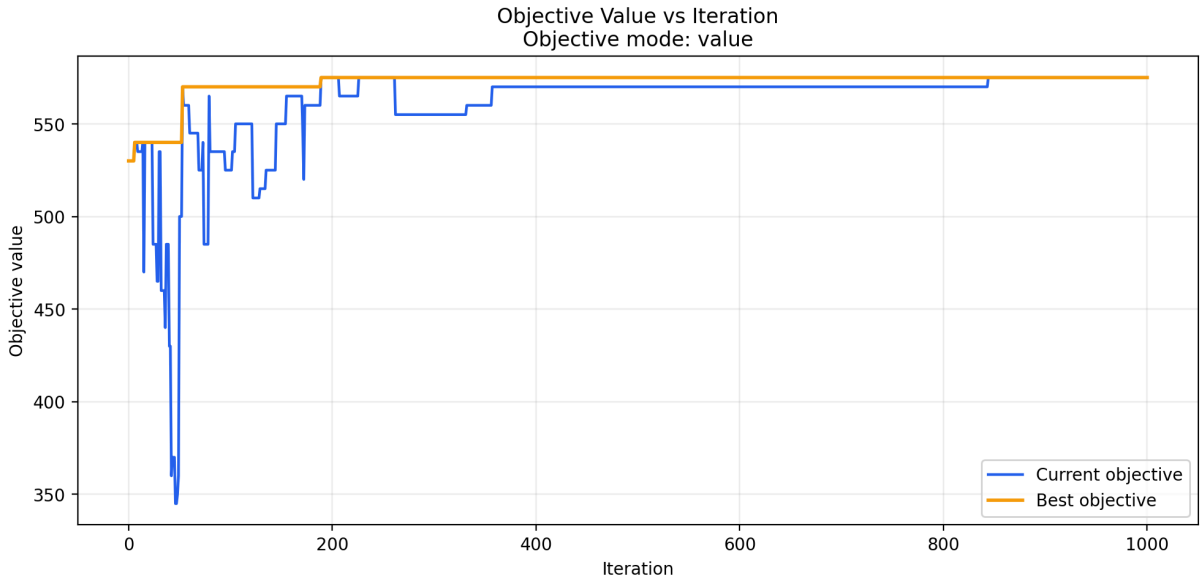


Gambar final state pada algoritma simulated annealing

Visualisasi lainnya tersedia pada direktori output/simulated_annealing.

Perubahan *Objective* terhadap Iterasi

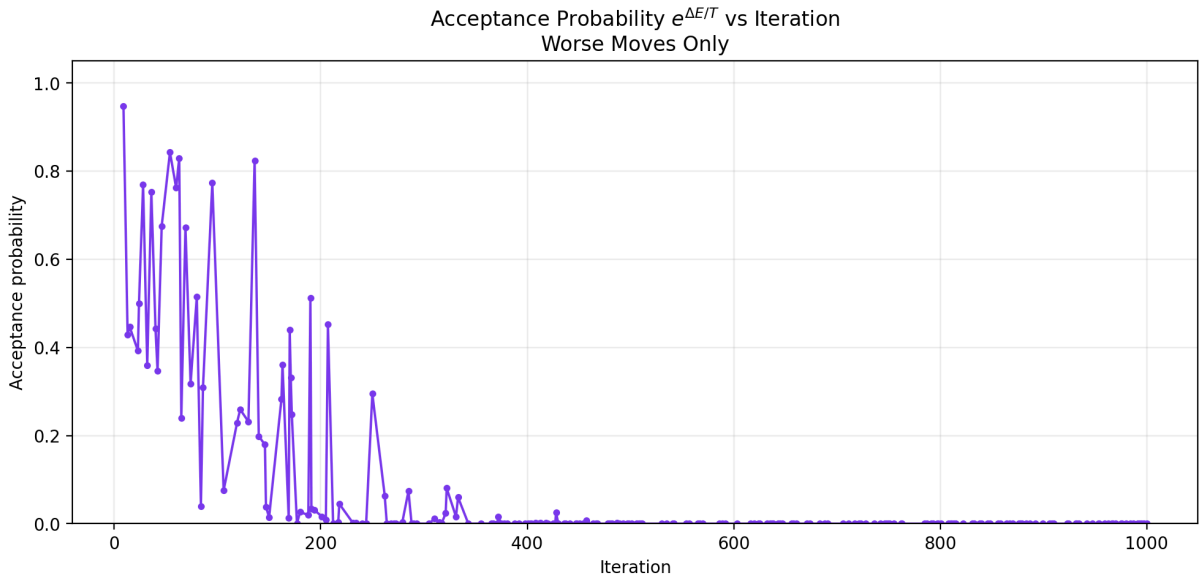
Grafik *objective* pada setiap percobaan menampilkan dua nilai, yaitu current objective dan best objective. *Current objective* menunjukkan nilai state yang sedang ditempati algoritma, sedangkan *best objective* menunjukkan nilai tertinggi yang pernah ditemukan sampai iterasi tersebut.



Pada awal pencarian, *current objective* mengalami kenaikan dan penurunan karena algoritma masih memiliki probabilitas yang cukup besar untuk menerima neighbor dengan nilai lebih rendah. Sementara itu, *best objective* tidak mengalami penurunan karena hanya diperbarui ketika ditemukan solusi yang lebih baik. Pada bagian akhir pencarian, perubahan *current objective* menjadi semakin kecil seiring menurunnya temperatur.

Probabilitas Penerimaan *Worse Move*

Sebagai informasi tambahan yang menggambarkan karakteristik Simulated Annealing, program menghasilkan grafik probabilitas penerimaan untuk *neighbor* dengan $\Delta E < 0$.



Pada iterasi awal, probabilitas penerimaan *worse move* masih relatif tinggi. Nilai tersebut kemudian semakin mendekati nol seiring penurunan temperatur. Informasi menunjukkan bahwa algoritma benar-benar melakukan eksplorasi melalui penerimaan solusi yang lebih buruk sebelum semakin selektif pada temperatur rendah.

Frekuensi *Stuck* pada Local Optima

Frekuensi *stuck* dihitung ketika *best objective* tidak mengalami peningkatan selama minimal 50 iterasi dan pemeriksaan terhadap maksimal 50 *neighbor* tidak menemukan solusi yang lebih baik. Pada mode value, Run 1 dan Run 2 masing-masing terdeteksi *stuck* dua kali, sedangkan Run 3 terdeteksi *stuck* satu kali. Dengan demikian, rata-rata frekuensi *stuck* pada tiga percobaan adalah 1,67 kali per run. Detail iterasinya telah ditampilkan pada tabel hasil utama.

Hasil Objective Tambahan (eta_weighted)

Pengujian berikutnya menggunakan *objective* tambahan eta_weighted. Testcase dan seed dibuat sama dengan mode value, sehingga perbedaan hasil berasal dari fungsi objective yang digunakan. Karena fungsi ini mempunyai skala yang berbeda dari objective value, nilai *objective* keduanya tidak dapat dibandingkan secara langsung. Oleh karena itu, tabel juga menampilkan total value pada state akhir.

Run	Seed	Objective Awal	Objective Akhir	Objective Terbaik	Total Value Akhir	Accepted Worse Moves	Stuck	Durasi
1	42	663.393	740.268	749.583	575	16	1	0.963
2	123	668.393	760.833	760.833	560	11	1	10.732
3	999	708.851	760.833	775.875	560	14	1	7.890
Rata-rata		**680.212**	**753.978**	**762.097**	**565**	**1**	**1**	**6.528**

State yang diperoleh adalah sebagai berikut.

Run	State Awal	State Akhir
1	C, F, G, H, I, J	A, B, F, G, I, L
2	C, D, F, H, J	A, C, D, G, I
3	A, C, E, F, G, H, L	A, C, D, G, I

State awal pada masing-masing *seed* sama dengan mode value, tetapi beberapa state akhir berbeda karena eta_weighted memberikan preferensi tidak hanya terhadap value paket tetapi juga ETA-nya. Perbedaan pengaruh kedua *objective* ini dibahas lebih lanjut pada bagian analisis.

Pengujian Bonus *Multi-Truck*

Simulated Annealing juga diuji terhadap fitur bonus *multi-truck*, di mana program menerima lebih dari satu truk dan memaksimalkan total value yang dapat diangkut seluruh armada.

Tiga skenario *multi-truck* yang digunakan adalah:

1. Dua truk identik, masing-masing berukuran 4 x 4 x 3 dengan kapasitas 18.
2. Dua truk heterogen, yaitu truk 3 x 6 x 3 berkapasitas 12 dan truk 5 x 3 x 4 berkapasitas 20.
3. Tiga truk dengan kapasitas terbatas, seluruhnya berukuran 4 x 4 x 3 dengan kapasitas masing-masing 8, 10, dan 12.

Masing-masing testcase menggunakan 14 paket dan dijalankan tiga kali dengan parameter SA serta *seed* yang sama seperti eksperimen utama. Berikut merupakan hasil percobaan dengan menggunakan mode value.

Skenario	***Objective* Akhir**			
	Seed 42	**Seed 123**	**Seed 999**	**Rata-Rata**
1	745	735	705	728.33
2	690	675	675	680.00
3	955	980	930	955.00

Pada seluruh skenario tersebut, *objective* dihitung sebagai total value paket yang dimuat pada seluruh truk. Visualisasi state awal, state akhir, dan riwayat objective masing-masing percobaan tersedia pada subdirektori *test case* yang bersesuaian di output/simulated_annealing/value/.

Eksperimen eta_weighted untuk ketiga *test case multi-truck* juga tersedia pada repository, tetapi hasil tersebut tidak digunakan sebagai pengukuran utama bonus *multi-truck* karena skala *objective*-nya berbeda dari total value sebagai *objective* utama.

2.2.3. Hasil Eksperimen Genetic Algorithm

Pengujian Genetic Algorithm dilakukan menggunakan satu truk berukuran 8 x 8 x 5, kapasitas berat 40, dan 12 paket. Eksperimen dibagi menjadi dua kelompok: variasi jumlah iterasi dengan populasi tetap dan variasi jumlah populasi dengan iterasi tetap. Setiap konfigurasi dijalankan tiga kali dengan seed 42, 123, dan 999. Sehingga total terdapat 18 percobaan.

Objective awal merupakan fitness maksimum populasi awal, sedangkan objective akhir merupakan fitness maksimum pada generasi terakhir. Durasi mencakup pembentukan populasi awal dan proses evolusi.

Variasi jumlah iterasi

Pada kelompok ini, jumlah populasi ditetapkan sebesar 20 individu, sedangkan jumlah iterasi divariasikan menjadi 50, 100, dan 200.

Iterasi	Seed	Objective awal	Objective akhir	Rata-rata fitness akhir	Durasi (s)
50	42	575	550	505,00	5,612
50	123	585	530	525,25	5,711
50	999	580	555	555,00	5,699
100	42	575	550	550,00	6,372
100	123	585	540	534,50	6,251
100	999	580	555	537,50	6,220
200	42	575	550	550,00	7,482
200	123	585	525	494,25	7,464
200	999	580	555	550,25	7,149

Variasi jumlah populasi

Pada kelompok ini, jumlah iterasi ditetapkan sebesar 100 generasi, sedangkan jumlah populasi divariasikan menjadi 10, 20, dan 50 individu.

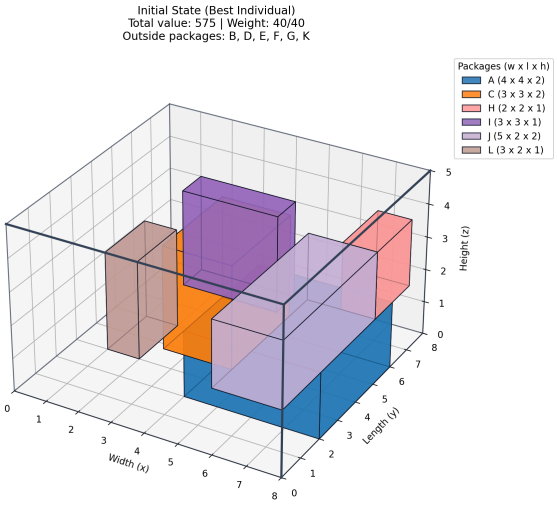
Populasi	Seed	Objective awal	Objective akhir	Rata-rata fitness akhir	Durasi (s)

10	42	575	540	514,50	3,180
10	123	570	530	530,00	3,046
10	999	580	535	519,00	2,928
30	42	590	560	560,00	9,460
30	123	585	505	505,00	8,823
30	999	580	550	550,00	9,318
50	42	590	565	460,40	16,395
50	123	585	515	515,00	15,019
50	999	580	560	546,40	15,375

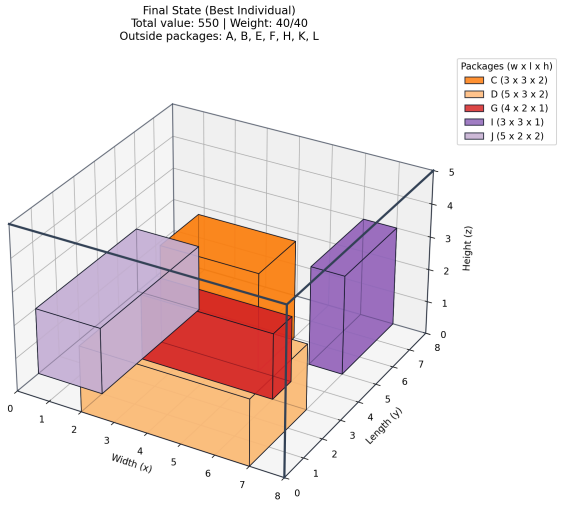
Visualisasi hasil eksperimen

Visualisasi setiap percobaan terdiri atas state awal, state akhir, serta grafik fitness maksimum dan rata-rata populasi terhadap generasi. Generasi ke-0 menunjukkan populasi awal sebelum proses evolusi.

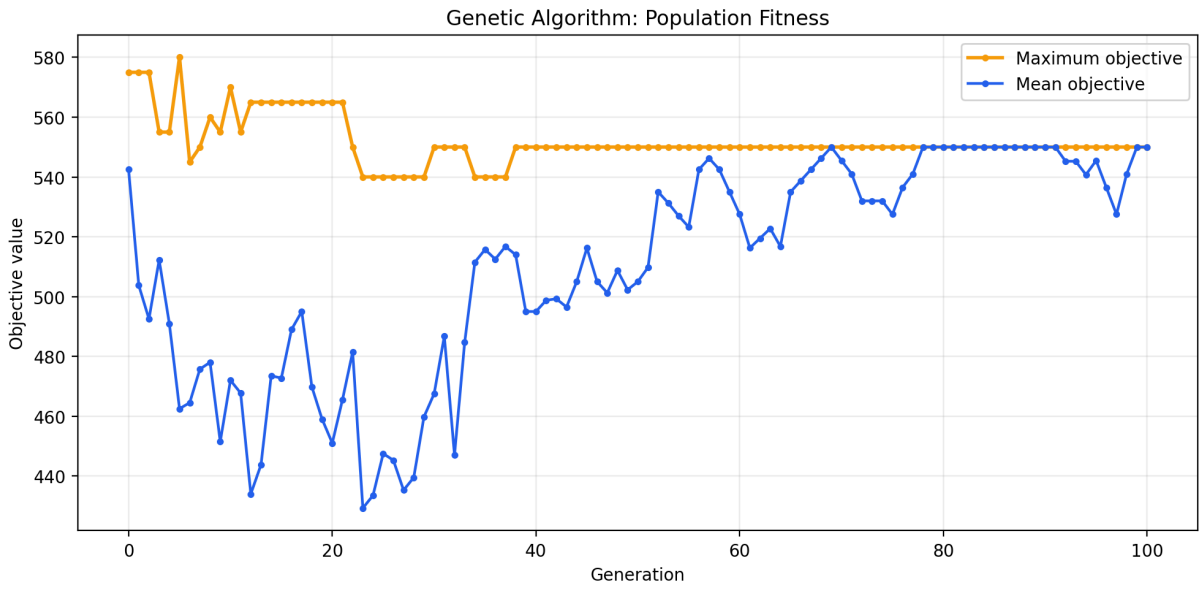
Sebagai contoh, pada konfigurasi populasi 20, iterasi 100, dan seed 42, objective awal sebesar 575 berubah menjadi 550 pada generasi terakhir.



Gambar initial state pada algoritma genetic algorithm



Gambar finall state pada algoritma genetic algorithm



Visualisasi percobaan lainya disertakan pada folder `output/genetic_algorithm`.

Pengujian *multi-truck*

Pengujian bonus multi-truck dilakukan secara terpisah menggunakan tiga skenario, masing-masing dengan tiga seed. Seluruh percobaan menggunakan populasi 20 dan 100 generasi. Objective dihitung sebagai total value paket yang dimuat oleh seluruh armada.

Skenario	Seed 42	Seed 123	Seed 999	Rata-rata objective akhir
Dua truk identik	650	745	685	693,33
Dua truk berbeda ukuran dan kapasitas	630	670	675	658,33
Tiga truk dengan kapasitas berbeda	940	850	855	881,67

2.3. Analisis dan Perbandingan

2.3.1. Perbandingan Nilai Objective Function

2.3.1.1. Hill-Climbing

Pada eksperimen dengan tiga seed yang sudah ditentukan pada semua algoritma, Steepest-Ascent, Stochastic, dan Sideways Move memberikan nilai objective akhir yang sama, yaitu 575, 560, dan 590. Kesamaan ini dibuat bukan berdasarkan proses iterasi yang identik antara satu sama lain. Pada steepest-Ascent, iterasi dilakukan dengan mengevaluasi semua neighbor valid dan segera berhenti ketika tidak terdapat peningkatan. Pada Stochastic hanya memeriksa satu neighbor acak setiap iterasi sehingga tetap berjalan hingga batas 100 iterasi pada seluruh run. Pada Sideways Move dapat berpindah pada objective yang sama dan seluruh run menggunakan batas maksimum 10 sideways move sebelum berhenti.

Berbeda dengan algoritma sebelumnya, Random-Restart memberikan rata-rata nilai objective akhir tertinggi daripada algoritma lainnya, yaitu 585,00, dan variansi hasil paling kecil pada tiga pengulangan. Hasil ini muncul karena algoritma memperoleh beberapa state awal yang berbeda sehingga memiliki lebih banyak kesempatan untuk keluar dari local optimum yang ditemui oleh satu kali pencarian. Nilai terbaik 590 dicapai pada seed 999. Nilai tersebut juga merupakan objective tertinggi yang diperoleh oleh varian lain pada eksperimen ini.

2.3.1.2. Simulated Anneling

Pada tiga percobaan dengan *objective value*, Simulated Annealing menghasilkan objective akhir sebesar 575, 575, dan 560, sehingga rata-ratanya adalah 570,00. Dibandingkan dengan nilai optimum 590, rata-rata tersebut mencapai sekitar 96,61% dari optimum. Dengan demikian, pada ketiga *seed* yang digunakan, Simulated Annealing mampu menghasilkan solusi yang relatif dekat dengan optimum, meskipun tidak mencapai nilai optimum. Selain itu, *best objective* yang pernah ditemukan masing-masing sebesar 575, 575, dan 585, dengan rata-rata 578,33. Perbedaan antara *final objective* dan *best objective* ini menunjukkan karakteristik Simulated Annealing yang dapat meninggalkan solusi terbaik yang pernah ditemukan karena masih memungkinkan penerimaan *worse move*.

Dibandingkan Steepest-Ascent Hill-Climbing, rata-rata objective akhir Simulated Annealing sebesar 570,00 sedikit lebih rendah daripada 575,00 yang diperoleh Steepest-Ascent. Ketiga varian Hill-Climbing lain memberikan rata-rata 575,00 untuk Stochastic dan Sideways Move, sedangkan Random Restart memperoleh

rata-rata tertinggi sebesar 585,00. Hal ini menunjukkan bahwa kemampuan menerima state yang lebih buruk tidak selalu menghasilkan objective akhir yang lebih tinggi. Pada testcase ini, Hill-Climbing dapat memanfaatkan improving neighbor secara langsung, sementara Random Restart memperoleh keuntungan tambahan dari beberapa titik awal pencarian.

Dibandingkan Genetic Algorithm, konfigurasi GA dengan rata-rata objective akhir terbaik, yaitu populasi 20 dan 100 generasi, memperoleh rata-rata 548,33. Dengan rata-rata 570,00, Simulated Annealing lebih tinggi sebesar sekitar 21,67 unit value pada testcase yang sama. Salah satu faktor yang memengaruhi hasil GA adalah penggantian populasi tanpa elitisme, sehingga individu terbaik tidak selalu dipertahankan pada generasi berikutnya. Sebaliknya, SA melakukan perubahan terhadap satu current state secara berurutan dan tetap menyimpan best state yang pernah ditemukan. Meskipun demikian, hasil ini berlaku pada konfigurasi dan tiga seed yang digunakan sehingga tidak menunjukkan bahwa SA selalu lebih baik daripada GA untuk seluruh kasus.

2.3.1.3. Genetic Algorithm

Pada eksperimen *single-truck*, konfigurasi GA dengan rata-rata objective akhir tertinggi adalah populasi 20 dan 100 generasi, yaitu 548,33. Nilai tersebut lebih rendah daripada rata-rata Steepest-Ascent Hill-Climbing sebesar 575,00 dan Simulated Annealing mode `value` sebesar 570,00. Dengan demikian, pada *testcase* dan konfigurasi yang diuji, GA belum memberikan kualitas hasil akhir yang lebih baik daripada kedua algoritma tersebut.

Untuk testcase ini, nilai optimum dapat ditentukan sebesar 590. Pemeriksaan seluruh $2^{12} = 4.096$ kombinasi paket menunjukkan bahwa total value tertinggi yang memenuhi kapasitas berat adalah 590. Karena terdapat penempatan valid yang mencapai nilai tersebut, batas atas itu juga merupakan optimum persoalan. Salah satu kombinasi yang mencapainya adalah paket A, C, E, G, H, I, dan L, dengan berat total 40.

Objective akhir tertinggi dari seluruh eksperimen GA adalah 565, diperoleh pada populasi 50, 100 generasi, dan seed 42. Hasil ini mencapai sekitar 95,76% dari optimum, dengan selisih 25 unit value.

Pada beberapa percobaan, populasi awal GA sebenarnya sudah memiliki individu bernilai 590, tetapi individu tersebut tidak bertahan sampai generasi terakhir. Kondisi ini dimungkinkan oleh penggantian populasi tanpa elitisme. Seleksi

proporsional terhadap fitness tidak menjamin individu terbaik selalu terpilih, sedangkan crossover, mutasi, dan repair dapat menghasilkan susunan dengan value lebih rendah.

2.3.2. Perbandingan Waktu Eksekusi dan Konsistensi

2.3.2.1. Hill-Climbing

Berdasarkan proses pengujian pada ketiga seed tersebut, Stochastic Hill-Climbing memiliki rata-rata waktu eksekusi paling singkat, yaitu sekitar 0,014 detik. Hal ini disebabkan algoritma hanya membangkitkan dan mengevaluasi satu neighbor acak pada setiap iterasi. njutkan pencarian pada state dengan objective yang sama. Random-Restart Hill-Climbing membutuhkan rata-rata waktu terlama, yaitu 1,530 detik, karena menjalankan Steepest-Ascent sebanyak enam kali dalam satu run, yang terdiri atas satu pencarian awal dan lima restart.

Dari sisi konsistensi, Random-Restart Hill-Climbing menghasilkan simpangan baku objective akhir paling kecil, yaitu 4,08, dengan rata-rata objective akhir 585,00. Sementara itu, tiga varian lain memiliki rata-rata objective akhir 575,00 dengan simpangan baku 12,25. Hasil ini menunjukkan bahwa penggunaan beberapa state awal pada Random-Restart membantu mengurangi ketergantungan terhadap satu konfigurasi awal dan meningkatkan peluang memperoleh solusi bernilai tinggi. Namun, kesimpulan tersebut hanya didasarkan pada tiga pengulangan sehingga belum cukup untuk menggambarkan kestabilan algoritma secara umum. Selain itu, waktu eksekusi dapat berubah bergantung pada perangkat dan kondisi saat program dijalankan.

2.3.2.2. Simulated Annealing

Durasi Simulated Annealing untuk tiga percobaan mode value adalah 9,862 detik, 4,657 detik, dan 23,289 detik, dengan rata-rata 12,603 detik. Nilai tersebut lebih tinggi dibandingkan Steepest-Ascent Hill-Climbing yang rata-rata membutuhkan sekitar 0,217 detik, Sideways Move 1,213 detik, Random Restart 1,530 detik, maupun konfigurasi GA populasi 20 dan 100 generasi yang membutuhkan rata-rata 6,281 detik. Stochastic Hill-Climbing pada eksperimen ini bahkan hanya membutuhkan sekitar 0,014 detik.

Waktu pencarian SA yang lebih besar terutama disebabkan oleh proses yang tetap berjalan hingga 1000 iterasi pada ketiga percobaan. Pada setiap iterasi, program membangkitkan dan memvalidasi *neighbor* baru, sedangkan ketika kondisi *stuck* terdeteksi dilakukan pemeriksaan tambahan terhadap hingga 50 *neighbor*. Selain

itu, generate_neighbor() dapat melakukan beberapa percobaan sebelum memperoleh konfigurasi yang valid. Karena tingkat kesulitan menemukan *neighbor* valid bergantung pada konfigurasi *current state*, waktu setiap iterasi tidak selalu sama. Hal tersebut menjelaskan perbedaan durasi yang cukup besar meskipun ketiga percobaan menggunakan jumlah iterasi dan parameter yang sama.

Dari sisi konsistensi *objective* akhir, tiga hasil SA sebesar 575, 575, dan 560 memiliki simpangan baku sampel sekitar 8,66. Nilai ini lebih kecil dibandingkan Steepest-Ascent, Stochastic, dan Sideways Move yang masing-masing memiliki simpangan baku 15,00, tetapi lebih besar dibandingkan Random Restart sebesar 5,00. Untuk konfigurasi GA populasi 20 dan 100 generasi, simpangan bakunya adalah sekitar 7,64. Dengan demikian, hasil akhir SA pada tiga percobaan relatif stabil.

2.3.2.3. Genetic Algorithm

Rata-rata durasi GA berkisar antara 3,051–15,596 detik, bergantung pada jumlah populasi dan generasi. Konfigurasi dengan rata-rata objective akhir tertinggi, yaitu populasi 20 dan 100 generasi, membutuhkan rata-rata 6,281 detik. Pada eksperimen yang dilakukan, durasi tersebut lebih lama daripada Steepest-Ascent Hill-Climbing sebesar 0,390 detik, tetapi lebih sebentar daripada Simulated Annealing sebesar 12,603 detik.

Perbandingan waktu tersebut karena cakupan pengukurannya berbeda. Durasi GA mencakup pembentukan populasi awal dan evolusi, sedangkan durasi Steepest-Ascent dan SA dihitung setelah state awal tersedia.

Konsistensi GA juga berbeda antarparameter. Pada 100 generasi, populasi 10 menghasilkan simpangan baku objective akhir sebesar 5,00, sedangkan populasi 30 dan 50 menghasilkan simpangan baku masing-masing 29,30 dan 27,54. Populasi 10 lebih konsisten pada tiga pengulangan ini, tetapi rata-rata objective akhirnya lebih rendah.

Oleh karena itu, konsistensi perlu dinilai bersama kualitas hasil. Simpangan baku kecil menunjukkan hasil yang berdekatan, tetapi tidak menjamin hasil tersebut mendekati optimum. Tiga pengulangan juga belum cukup untuk menyimpulkan kestabilan algoritma secara umum.

2.3.3. Pengaruh Parameter Genetic Algorithm

Ringkasan hasil setiap konfigurasi ditunjukkan pada tabel berikut. Simpangan baku yang digunakan merupakan simpangan baku dari tiga pengulangan.

Populasi	Generasi	Rata-rata objective akhir	Simpangan baku	Rata-rata durasi (s)
20	50	545,00	13,23	5,674
20	100	548,33	7,64	6,281
20	200	543,33	16,07	7,365
10	100	535,00	5,00	3,051
30	100	538,33	29,30	9,200
50	100	546,67	27,54	15,596

Pengaruh jumlah generasi

Pada populasi tetap 20, penambahan generasi dari 50 menjadi 100 meningkatkan rata-rata objective akhir dari 545,00 menjadi 548,33. Namun, pada 200 generasi, nilainya menurun menjadi 543,33. Durasi terus meningkat. Hasil ini menunjukkan bahwa penambahan generasi tidak selalu meningkatkan kualitas solusi akhir pada implementasi tanpa elitisme.

Pengaruh jumlah populasi

Pada kelompok variasi populasi dengan 100 generasi, rata-rata objective akhir meningkat dari 535,00 pada populasi 10 menjadi 538,33 pada populasi 30 dan 546,67 pada populasi 50. Namun, peningkatan tersebut diikuti kenaikan waktu eksekusi karena lebih banyak individu perlu dibentuk, diperbaiki, dan dievaluasi.

2.3.4. Perilaku Pencarian Simulated Annealing

Pada mode value, frekuensi stuck yang terdeteksi pada tiga percobaan adalah 2, 2, dan 1 kali, dengan rata-rata 1,67 kali per run. Run 1 terdeteksi stuck pada iterasi 155 dan 239, sedangkan best objective 575 pertama kali diperoleh pada sekitar iterasi 189. Hal ini menunjukkan bahwa setelah kondisi local optimum pertama terdeteksi, algoritma masih berhasil berpindah dari daerah tersebut dan menemukan solusi dengan objective yang lebih tinggi. Pola serupa terjadi pada Run

2, di mana kondisi stuck pertama terdeteksi pada iterasi 179, sedangkan best objective 575 baru ditemukan pada sekitar iterasi 259.

Kemampuan tersebut berkaitan dengan penerimaan worse move. Pada Run 1, Run 2, dan Run 3 masing-masing terdapat 20, 16, dan 15 perpindahan menuju state yang lebih buruk yang tetap diterima. Penerimaan tersebut memungkinkan nilai *objective current state* bergerak turun sebelum kembali mencari daerah dengan nilai yang lebih tinggi. Konsep ini membedakan SA dari Hill-Climbing biasa yang hanya bergerak menuju state yang lebih baik.

Pengaruh proses pendinginan juga terlihat dari distribusi *worse move*. Pada ketiga percobaan mode value, *worse move* terakhir yang diterima terjadi masing-masing sekitar pada iterasi 262, 239, dan 353. Setelah bagian tersebut, hampir seluruh perpindahan yang lebih buruk ditolak karena temperatur semakin rendah. Temperatur dimulai dari 100, dan dengan cooling_rate = 0,99, turun menjadi sekitar 0,00432 setelah 1000 iterasi. Dengan demikian, perilaku algoritma berubah dari lebih eksploratif pada awal pencarian menjadi lebih selektif pada bagian akhir, sesuai prinsip bahwa probabilitas penerimaan state yang lebih buruk menurun seiring penurunan temperatur.

Run 3 menunjukkan keterbatasan lain dari mekanisme tersebut. *Initial state* pada run ini sudah memiliki objective 585, yang merupakan nilai terbaik sepanjang pencarian, tetapi *final state* hanya memiliki *objective* 560. Algoritma melakukan eksplorasi dengan menerima beberapa state yang lebih buruk, tetapi tidak berhasil menemukan solusi yang melebihi *initial state* sebelum pencarian berakhir. Kasus ini menunjukkan bahwa kemampuan keluar dari local optimum memberikan peluang untuk menemukan daerah pencarian yang lebih baik, tetapi berisiko berakhir di *state* dengan nilai *objective* yang lebih rendah dari *best state*.

Pada konfigurasi yang digunakan, seluruh *run* berhenti karena mencapai max_iterations, bukan karena temperatur mencapai min_temperature. Hal ini menunjukkan bahwa batas 1000 iterasi menjadi faktor penghentian utama dengan parameter seperti yang digunakan.

Kesimpulan dan Saran

3.1. Kesimpulan

Berikut adalah kesimpulan yang dapat dirumuskan setelah mengerjakan tugas besar ini:

- Algoritma Hill-Climbing, Simulated Annealing, dan Genetic Algorithm berhasil diterapkan sebagai algoritma *local search* untuk mencari susunan paket dengan mempertimbangkan kapasitas berat, batas kontainer, overlap, support, dan sifat fragile.
- Pada testcase dan seed yang digunakan, Random-Restart Hill-Climbing menghasilkan rata-rata objective akhir tertinggi, yaitu 585,00, mendekati nilai optimum testcase sebesar 590. Hal ini sejalan dengan tujuan algoritma Random-Restart Hill-Climbing itu sendiri, yaitu mencari global optimum. Pada uji coba yang kami lakukan kami tidak dapat menghasilkan local optimum, dikarenakan batas iterasi yang masih kurang.
- Simulated Annealing menghasilkan rata-rata objective akhir 570,00. Penerimaan solusi yang lebih buruk membantu eksplorasi, tetapi tidak menjamin state akhir memiliki nilai yang sama dengan solusi terbaik yang pernah ditemukan.
- Konfigurasi GA dengan rata-rata objective akhir tertinggi adalah populasi 20 dan 100 generasi, yaitu 548,33. Penambahan populasi atau generasi tidak selalu meningkatkan kualitas hasil. Penggantian populasi tanpa elitisme juga memungkinkan hilangnya individu terbaik.

3.2. Saran

Berikut adalah saran untuk pengembangan lebih lanjut:

- Memperluas pengujian menggunakan lebih banyak seed dan variasi testcase, termasuk jumlah paket, dimensi, kapasitas, proporsi paket fragile, serta konfigurasi armada.
- Melakukan eksplorasi parameter lebih lanjut untuk mencari keseimbangan antara kualitas solusi, konsistensi, dan waktu eksekusi.
- Menyeragamkan cakupan pengukuran waktu, kondisi perangkat, dan metode perhitungan simpangan baku agar perbandingan antaralgoritma lebih konsisten.

Referensi

[1] M. L. Khodra, "Beyond Classical Search: Classical vs Local Search," YouTube, 2021. [Video]. Available: Beyond Classical Search: Classical vs Local Search [Accessed: Sep. 16, 2026].

[2] M. L. Khodra, "Beyond Classical Search: State (Value, Neighbor)," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=_6rla37QorM. [Accessed: Sep. 16, 2026].

[3] M. L. Khodra, "Beyond Classical Search: Hill-climbing Search," YouTube, 2021. [Video]. Available: https://www.youtube.com/watch?v=L0b0CN3Fds8. [Accessed: Sep. 16, 2026].

[4] M. L. Khodra, "Beyond Classical Search: Simulated Annealing," YouTube, 2021. [Video]. Available: Beyond Classical Search: Simulated Annealing [Accessed: Sep. 16, 2026].

[5] M. L. Khodra, "Beyond Classical Search: Genetic Algorithm," YouTube, 2021. [Video]. Available: Beyond Classical Search: Genetic Algorithm [Accessed: Sep. 16, 2026].

[6] Genetic Algorithm, Geeksforgeeks (Genetic Algorithms - GeeksforGeeks)

[7] Ayachi, Kammarti, Ksouri, Borne. A Genetic algorithm to solve the container storage space allocation problem. International conference on Computational Intelligence and Vehicular System (CIVS). 2010. 1303.1051.

Lampiran

Penggunaan Gen-AI

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan (level 1-5)
Pemeriksaaan Ejaan: menggunakan tools seperti Grammarly, Deepl, Quilbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya. **Sebutkan tool yang digunakan:**	Tidak	1
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya. **Sebutkan tool yang digunakan:** ChatGPT	Ya	2
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni Ai, Zoom-Companion, atau sejenisnya. **Sebutkan tool yang digunakan:** ChatGPT	Ya	2
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffuson, Microsoft Designer, Gemini, Canva AI, atau sejenisnya. **Sebutkan tool yang digunakan:**	Tidak	1

Pembagian Tugas

NIM	Nama	Tugas
13524006	Gabriella Botimada Lubis	- Implementasi algoritma Simulated Annealing - Membuat visualisasi hasil - Mengerjakan laporan
13524038	An Dafa Anza Avansyah	- Implementasi Genetic Algorithm - Implementasi bonus *multi-truck* - Mengerjakan laporan

13524059	Raymond Jonathan Dwi Putra J.	- Implementasi algoritma Hill-Climbing semua varian - Membuat pondasi program - Mengerjakan laporan